



Faculty of Science and Engineering

Microprocessors Spring 2026

Smart Home System Report

Jana Abu fodeh

Hafsa Bakhtawar Bila

Rayan Radwan

Contents

Introduction.....	2
Procedure	3
Phase A: Software Development and Simulation.....	3
Phase B: Physical Hardware Assembly & Deployment	3
Hardware and peripherals	4
Peripheral Initialization.....	5
System Block Diagram & Flowchart.....	6
Testing Methodology and Results.....	6
Discussion	8
Results	8
Conclusion.....	9
Additional Information	9

Introduction

The Smart Home Control Hub is an embedded system developed using the PIC16F877A microcontroller to demonstrate the integration of multiple hardware peripherals into one practical application. The system continuously monitors the surrounding environment using different sensors and reacts automatically by controlling LEDs, a relay-driven fan, and emergency indicators.

The project combines several built-in peripherals of the PIC16F877A, including GPIO, ADC, PWM, I2C, USART, and external interrupts. An SH1106 OLED display provides live feedback to the user, while an HC-05 Bluetooth module allows remote communication with a laptop through serial commands.

The final implementation focuses on environmental monitoring and emergency handling. Room brightness is measured using an LDR sensor, smoke concentration is monitored using an MQ-2 sensor, and fire hazards are detected using a flame sensor. In addition, a dedicated emergency push button demonstrates the use of hardware interrupts for immediate system response.

Procedure

To ensure continuous debugging and a stable workflow, the project follows a strict step-by-step methodology progressing from initial software simulation to physical hardware deployment. And to reduce integration problems, the project was implemented incrementally. Each module was tested individually before combining all peripherals into one final system.

Phase A: Software Development and Simulation

1. **Toolchain Setup:** Code is written and compiled in C using Microchip's **MPLAB X IDE** and the **XC8 compiler**. Individual peripherals were *first* verified in Proteus simulation before physical implementation. The compiled .hex file is loaded directly into **Proteus Design Suite 8** for full virtual circuit testing.
2. The OLED display was initialized through software I2C communication and tested by displaying startup messages.
3. Analog sensor values were *simulated* using potentiometers to verify ADC conversion and scaling.
4. PWM output was connected to LEDs to confirm brightness control.
5. USART communication was tested using the Proteus Virtual Terminal before connecting the HC-05 Bluetooth module.
6. The interrupt routine was verified by triggering the external interrupt input and confirming immediate execution.

Phase B: Physical Hardware Assembly & Deployment

1. **Component Inventory & Inspection:** All primary sensors, modules, and passive components are gathered and checked based on the verified design checklist.
2. **Breadboard Prototyping:** Components are initially laid out onto solderless breadboards to avoid premature soldering errors. The PIC16F877A, its 20 MHz crystal oscillator, and decoupling capacitors are placed to create a stable hardware core.
3. The SH1106 **OLED** display was connected through RC3 (SCL) and RC4 (SDA) using 10 kΩ pull-up resistors. The **flame sensor** digital output was connected to RB2.
4. The **LDR** voltage divider was connected to analog channel AN1. The **MQ-2 smoke sensor** analog output was connected to AN2 while its digital output was connected to RB3.
5. The **HC-05 Bluetooth module** was connected to the USART interface using RC6 and RC7 with a voltage divider protecting the module RX pin. The relay module, buzzer, LEDs, emergency button, and recovery button were connected to their assigned **GPIO** pins.

6. **System Flash & Testing:** The final verified firmware code is flashed directly onto the physical PIC16F877A chip. The system is run under a real load to verify physical relay switching, true analog sensor tolerances, and fast interrupt processing.

Hardware and peripherals

The Smart Home Control Hub demonstrates the integration of **almost all** required PIC16F877A peripherals. The architectural framework splits responsibilities across specialized internal peripherals within the PIC and specific external hardware components:

Microcontroller Internal Peripherals

- ◆ **GPIO (General-Purpose Input/Output):** GPIO pins are configured as digital inputs and outputs for the flame sensor, smoke digital output, relay module, buzzer, emergency button, recovery button, and status LEDs.
- ◆ **ADC (Analog-to-Digital Converter):** The 10-bit ADC converts analog voltages into digital values.
 - AN1 reads the LDR voltage divider and converts the 0–5 V input into a 10-bit value (0–1023), which is mapped to a 0–100% brightness percentage.
 - AN2 reads the MQ-2 analog output to estimate smoke concentration and display it on the OLED.
- ◆ **PWM (Pulse Width Modulation):** The CCP1 PWM module drives three LEDs whose brightness changes automatically according to ambient light conditions. CP1 hardware module on RC2 generates a 4 kHz PWM signal. The duty cycle is inversely proportional to the LDR reading so that darker environments produce brighter LEDs. Employs the internal Capture/Compare/PWM (CCP) hardware module to alter the duty cycle delivered to the interior LED strip for ambient lighting adjustments.
- ◆ **I2C Protocol:** Acts as a 2-wire serial master bus to concurrently control data flow. The display continuously shows sensor readings and system status. A software (bit-banged) I2C master is implemented on RC3 (SCL) and RC4 (SDA), with 10 kΩ pull-up resistors required to keep the bus lines in their idle HIGH state.

- ◆ **USART (Universal Synchronous Asynchronous Receiver-Transmitter):** USART communication at serial 9600 baud (SPBRG=32 at 20 MHz) provides wireless serial communication through the HC-05 Bluetooth module. Commands entered from PuTTY on a laptop are received by the PIC and used to control the relay. RC6 transmits confirmation strings back to laptop. RC7 receives text commands from HC-05 Bluetooth module. Voltage divider on TX line protects HC-05 3.3V RXD input

- ◆ **EEPROM (Non-Volatile Memory):** EEPROM persistent storage was evaluated but deliberately excluded from this system. In a typical smart home sensor context, persisting sensor state across power cycles is not logically meaningful a flame detected before power-off does not imply a flame exists after power-on. Restoring a “fire detected “ state from EEPROM would produce false alarms and mislead users about current room conditions. User preferences in this system are managed through real-time Bluetooth commands (FAN ON / FAN OFF), which are re-issued by the user after power-on as needed. This design choice prioritizes system correctness over blind peripheral compliance

- ◆ **Hardware Interrupts:** INT0 on RB0 configured as rising-edge external interrupt. ISR sets emergency flag in microseconds regardless of main loop state, triggering buzzer, red LED, OLED emergency screen, and sensor freeze

The emergency push button is connected to RB0 (INT0). When pressed, the interrupt service routine immediately activates emergency mode regardless of the current execution point.

Peripheral Initialization

Before entering the main application loop, all peripherals are initialized.

- GPIO directions are configured using the TRIS registers.
- ADC is initialized for channels AN1 and AN2.
- CCP1 is configured for PWM output.
- Software I2C initializes communication with the OLED display.
- USART is configured for 9600 baud asynchronous communication.
- External interrupt INT0 is configured for rising-edge detection.
- Global and peripheral interrupts are enabled.

Table 1. Pin Assignment Table

Component	PIN	PIN #	Peripheral Used
OLED SCK	RC3	18	I2C
OLED SDA	RC4	23	
HC-05 TXD/ u2192 PIC RX	RC7	26	USART
HC-05 RXD/ u2190 PIC TX	RC6	Via divider , 25	
LDR analog	RA1 / AN1	3	ADC
MQ-2 AO	RA2/ AN2	4	ADC
MQ-2 DO	RB3	36	GPIO
Flame Sensor OUT	RB2	35	GPIO
Emergency Button	RB0 /INT0	33	Interrupt
Recovery Button	RB1	34	GPIO
PWM LEDs (x3)	RC2 / CCP1	17	PWM
Relay IN1 (fan)	RD7	30	GPIO
Buzzer	RD3	22	GPIO
Red Emergency LED	RD2	21	GPIO
Green Recovery LED	RD1	20	GPIO
Flame Indicator LED	RD6	29	GPIO
Crystal OSC1/OSC2	13 , 14	13 , 14	Clock
MCLR Pull-up (10k\ u03A9)	1	1	Reset

System Block Diagram & Flowchart

[Block diagram](#)

[Flowchart](#)

Testing Methodology and Results

Each subsystem was tested independently before full integration. Sensors like the flame sensor were tested without the PIC first to confirm functionality before programing it.

Table 2. Testing method and result

OLED Display	Display startup text and live values	Passed Initially blank, pull-up resistors were wired in series not to 5V column offset (+2) required
ADC (LDR)	Cover/uncover sensor Measure the voltage entering the PIC pin (lower LDR resistance higher voltage)	ADC values changed correctly and % displayed on the OLED 3.8V bright, 1.0V covered

MQ-2 Analog	Apply smoke source (<i>perfume</i> was used to <u>simulate smoke</u> and trigger the sensor safely)	Analog readings increased False triggers initially \u2014 sensor needed 60s warm-up.
MQ-2 Digital	Adjust threshold potentiometer to become more sensitive	Digital output switched correctly Potentiometer adjusted for correct threshold
Flame Sensor	Detect infrared source (measure voltage reading at the OUT pin <u>and simulate flame safely by</u> using IR waves from a remote control)	Active-low operation confirmed and LED flickered with each remote press
PWM	Change room brightness (the darker the room the brighter the LEDs , compare to the LDR brightness %) (test multiple speeds to find the smoothest brightness adjustment)	LED brightness adjusted smoothly , PR2 values set for 20 MHz 4kHz was found for PWM speed
PICkit 3 Programming	MPLAB X flash	Programming successful after fix, Invalid Device ID error PIC legs not fully seated. VDD reading 1.x V PICkit powering circuit, switched to external 5V supply. Unchecked Power from PICkit in project properties
USART	Send "FAN ON" and "FAN OFF" create a virtual COM using PuTTY , serial communication via COM5 and connect with the Bluetooth and send commands	Relay switched successfully SPBRG=129 gave wrong baud (2403 not 9600). Corrected to SPBRG=32. COM3 and COM5 both opened, tested both
Relay	Drive external fan , measure Voltage going to the fan	Operated correctly, fan receives 12v from the relay
Emergency Interrupt	Press emergency button , check if buzzer is activated and OLED shows trigger , COM5 displays emergency so no commands can activate the fan	ISR executed immediately, all triggers are on and COM5 displays emergency so no commands can activate the fan
Full Integration	Run all modules together after uploading the final code to the PIC and simulate cases to trigger the sensors	Stable operation confirmed

NOTE : Several resistors were used for current limiting and voltage dividing and as pull up resistors

Several debugging issues were encountered during development.

- ◆ The OLED initially failed to communicate because pull-up resistors were missing on the I2C lines. Adding 10 k Ω pull-ups solved the issue.
- ◆ USART baud rate calculations were corrected to achieve reliable Bluetooth communication. Different SPBRG causes failed communication.
- ◆ The flame sensor was verified to operate as an active-low device and the software logic was updated accordingly. Smoke DO is active low , when high there is no smoke , the logic in the code needed to be flipped.

- ◆ HC-05 communication was validated using PuTTY before integration with the rest of the system. Wrong COM number caused error , used COM was checked from device manager
- ◆ Wrong physical connection for the push buttons caused illogical behavior on push
- ◆ Floating MCLR caused errors in programming the PIC

Discussion

Integrating multiple peripherals on the PIC16F877A required careful management of timing and hardware resources. Since several modules operate simultaneously, blocking delays were minimized to prevent communication failures and display freezing.

The use of software I2C together with hardware USART allowed the OLED display and Bluetooth communication to operate independently. PWM brightness control was handled by the CCP hardware module, reducing processor workload.

The hardware interrupt provided one of the fastest responses in the system. Unlike polling methods, pressing the emergency button immediately transfers execution to the interrupt service routine, allowing the system to react within microseconds.

Throughout development, each module was tested independently before integration, making debugging significantly easier and reducing interaction problems between peripherals.

Results

The Smart Home Control Hub successfully met all functional goals and requirements across individual simulations and physical testing:

- ◆ **System Startup Validation:** Upon booting up, the system successfully runs all peripheral initializations and updates. The OLED display initializes correctly and continuously updates system information.
- ◆ **Automated Light & Climate Regulation:** The LDR measures ambient brightness and automatically adjusts LED intensity using PWM. The ADC scaling changes the PWM brightness dynamically between 0% and 100% depending on ambient lighting levels. The MQ-2 sensor reports smoke levels through both analog and digital outputs. The flame sensor detects fire conditions and activates emergency mode.

- ◆ **Schedule Execution & Remote Interface:** Bluetooth communication through the HC-05 successfully receives remote commands from PuTTY to control the relay-driven fan.”FAN ON” or “FAN OFF”
- ◆ **Hardware-Level Safety Performance:** manual panic button fires the external interrupt service routine (ISR) in microseconds. This immediately overrides the processor, shuts off the main relay loads, sounds the active buzzer, activates the red emergency LED, and locks the display panel onto a high-priority danger warning message. All settings remain intact through simulated power-cycling tests. A recovery button should be pressed to go back to the normal state.

The integrated system operated reliably during repeated hardware testing.

Conclusion

This project successfully demonstrated the integration of multiple PIC16F877A peripherals into a practical smart home application. GPIO, ADC, PWM, I2C, USART, and hardware interrupts were all implemented and verified through simulation and physical testing.

The system provides automatic lighting control, smoke and flame monitoring, Bluetooth-based remote fan control, and rapid emergency handling with OLED status feedback. The modular design also allows future expansion by adding features such as RTC-based scheduling or EEPROM-based configuration storage.

Additional Information

Hardware exact connections for documentation :

PIC Power & Clock

Pins 11, 32 → 5V | Pins 12, 31 → GND

MCLR pin 1 → 10kΩ → 5V

Crystal pins 13/14 with 22pF caps to GND

OLED (SH1106 I2C)

VCC → 5V | GND → GND

SCK → RC3 pin 18 (10kΩ pull-up to 5V)

SDA → RC4 pin 23 (10kΩ pull-up to 5V)

LDR

5V → LDR → junction → RA1 pin 3 | junction → 10kΩ → GND

MQ-2 Smoke Sensor

VCC → 5V | GND → GND

AO → RA2 pin 4 | DO → RB3 pin 36

Flame Sensor

VCC → 5V | GND → GND | OUT → RB2 pin 35

HC-05 Bluetooth

VCC → 5V | GND → GND

TXD → RC7 pin 26 (direct)

RXD → through divider: RC6 pin 25 → 1kΩ → junction → RXD, junction → 1kΩ → GND

Relay (Fan control)

VCC → 5V | GND → GND | IN1 → RD7 pin 30

COM → 12V(+) | NO → Fan(+) | Fan(-) → 12V GND

LEDs

PWM LEDs (×3): RC2 pin 17 → 330Ω → each LED → GND

Flame indicator LED: RD6 pin 29 → 330Ω → LED → GND

Status LED: RD0 pin 19 → 330Ω → LED → GND

Buzzer

Positive → RD3 pin 27 | Negative → GND

Emergency Button

One side → 5V | Other side → RB0 pin 33 + 10kΩ → GND

Recovery Button

One side → 5V | Other side → RB1 pin 34 + 10kΩ → GND

Red Emergency LED

RD2 pin 28 → 330Ω → LED → GND

Green Recovery LED

RD1 pin 22 → 330Ω → LED → GND

Sensor Logic and Implementation

LDR – Analog Light Sensing

The LDR forms a voltage divider with a 10 kΩ resistor. Bright light lowers the LDR resistance, raising the junction voltage. Dark conditions increase the resistance and lower the voltage. The ADC converts this 0–5 V range into a 10-bit value (0–1023), which is then mapped to a 0–100% brightness percentage. This percentage is displayed on the OLED and is also used to control the PWM LED brightness inversely, using the formula:

Duty Cycle = 1023 – LDR Reading

As a result, darker rooms produce brighter LEDs, while brighter rooms cause the LEDs to dim automatically.

MQ-2 – Smoke/Gas Detection

The MQ-2 module provides two outputs. The **AO (Analog Output)** pin generates a continuous voltage proportional to the detected gas or smoke concentration. This signal is read through ADC channel AN2 and displayed as a smoke percentage on the OLED.

The **DO (Digital Output)** pin uses an onboard comparator with an adjustable threshold set by the module's potentiometer. When the smoke level exceeds this threshold, the DO pin goes HIGH. This signal is read on RB3 and displayed on the OLED as a simple **YES/NO** smoke status.

The MQ-2 sensor requires approximately **60 seconds of warm-up time** after power-up before its readings become stable.

Flame Sensor – Active-Low IR Detection

The flame sensor contains a photodiode that is sensitive to infrared wavelengths in the **760–1100 nm** range. When infrared radiation from a flame is detected, the onboard comparator pulls the digital output LOW, making it an **active-low device**.

For this reason, the software checks:

```
FLAME_PIN == 0
```

to determine whether a flame has been detected.

A dedicated indicator LED connected to RD6 mirrors the flame detection status in real time. During testing, it was also observed that the sensor responds to TV and air conditioner remote controls because they emit infrared signals within the same wavelength range.

Hardware Interrupt – Emergency Button

The emergency push button is connected to **RB0**, the PIC16F877A's dedicated **INT0 external interrupt pin**, and is configured for **rising-edge detection**.

When the button is pressed, the hardware immediately interrupts the CPU regardless of what the program is currently executing—even if it is inside a delay or an I2C communication routine—and jumps directly to the Interrupt Service Routine (ISR).

The ISR sets an emergency flag and clears the interrupt flag within microseconds. The main program then detects this flag and enters a locked emergency state where:

- The buzzer is activated.
- The red emergency LED turns on.
- Sensor updates are temporarily frozen.
- The OLED switches to the emergency display.

A separate recovery button connected to **RB1** is monitored during this state and allows the system to return to normal operation when pressed.

Bluetooth – USART Fan Control

The HC-05 Bluetooth module creates a virtual COM port on the connected laptop using the Bluetooth Serial Port Profile (SPP). PuTTY opens this virtual COM port at **9600 baud**, allowing text commands to be sent wirelessly.

The HC-05 forwards the received bytes to the PIC's USART receive pin (**RC7**). The PIC stores incoming characters in a string buffer and, upon receiving a newline character, compares the completed string with predefined commands using `strcmp()`.

For example:

- **"FAN ON"** sets **RD7 HIGH**, activating the relay and closing the 12 V fan circuit.
- **"FAN OFF"** sets **RD7 LOW**, deactivating the relay and turning the fan off.

After executing the command, the PIC transmits a confirmation message back to the laptop through **RC6**, providing feedback to the user.